Learned Page Replacement in xv6

Week 16 update: kernel results against the classical policies

Summary

The question. When a program needs more memory than it is given, the kernel must repeatedly choose a page to throw out (evict). Can a choice learned from data beat the classical rules inside a real kernel (xv6), and is the extra work it does worth it? Everything in the main text was measured in xv6 running on QEMU unless it says "simulator". Numbers are page faults divided by Clock's faults on the same program (below 1 is better). Each workload's streams are averaged (geometric mean), then the 14 workloads count equally. Memory is 5, 10 or 20% of each program's own pages. Intervals are 95% intervals from resampling the workloads, then the streams inside them; bold blue means the whole interval is below 1.

What we found.

- **The learned linear score beats Clock** at 10% memory (0.914, interval 0.85 to 0.98); at 5% (0.976) and 20% (0.882) it is better on average but the intervals include 1.
- **It does not beat ARC.** Against ARC it is 1.031, 0.970 and 0.896 at 5, 10 and 20%; no interval excludes 1. ARC itself beats Clock at 5 and 10% (0.947, 0.943). **We have not shown that a learned policy beats the best classical policy on average.**
- **Heavier models are not better than the linear score.** At 10% memory (74 streams) the CNN, deeper MLP and set-attention are level with it (1.000, 0.998, 0.992); boosted trees (1.061) and ranking trees (1.476) are worse. They cost 20 to 150 times as many instructions.
- **Looking at fewer pages costs little.** Scoring 32 randomly chosen pages instead of all of them is level with scoring all (0.998), and 16 is 1.3% worse (1.013), at 10% memory on 74 streams. The cost then stops growing with memory size (Table 10). None of five other ways of choosing the pages beat random (simulator).
- **LeCaR**, a published learning policy, is worse than Clock in the form a kernel can run, at every memory size (1.051, 1.113 and 1.212 at 5, 10 and 20%; all intervals above 1).
- **Cost.** Choosing a victim with the linear score costs about 260 instructions per page looked at in the current kernel (16 800 with 68 pages, 134 000 with 516; 22 800 and 183 000 before the code change of Section 5.4, which keeps every victim). In time, that is about 1 to 1.5 µs per eviction on this PC's processor (provisional: timed while other jobs ran), small against a measured SSD read (63 µs) and tiny against a hard-disk read (8.9 ms), so on this PC it pays for itself. **But the instruction counts come from one synthetic benchmark**; a rough check on the real programs (Section 5.2) shows that this is fair for the linear score but underestimates ARC and LHD by up to about 2 times. Exact counts on the real programs are planned.

Corrections to earlier reports. (1) "Three DAgger rounds did not help" (first version of the study) is not reproduced: rerun with the current seeds, DAgger rescued one failing model and not others (Appendix C). (2) An earlier estimate that an evenly spread sample saves 5% of the instructions was wrong; measured, it saves nothing (Section 4). (3) The cost table used to show only the 68-page column; it now shows every size we measured.

1 What was run

Kernel runs. Each run is one program in xv6 with its resident memory limited, so every eviction goes through the policy under test. The program first sets itself up (its code, stack and other fixed pages, typically 4), then limits itself to those pages plus 5, 10 or 20% of the pages its data needs (for PageRank: 83, 167 or

333 of 1 667). xv6 runs are exactly repeatable: the same program, memory size and policy always give the same faults. 3 405 runs are merged in `report/results2/kernel_eval_all.csv`; they were split between the thesis PC and Kaggle notebooks (CPU only), and 180 runs repeated on both gave identical results.

Workloads (14). B-tree, graph (PageRank, BFS), key-value store (`kv`) and the same store driven by recorded Twitter traffic (`kv-T`), LZW compression, matrix multiply, sort, heap allocator churn, hash join, sparse matrix–vector product (SpMV), pointer chase, Bloom filter, a suite of synthetic access patterns (`pat`, from the program `patbench`) and four small application-shaped programs (`app`). 74 streams (runs of these programs with different settings and seeds), each at the three memory sizes. Models were trained only on separate training streams of the first six (btree, graph, kv, LZW, matmul, sort); the other eight workloads were never seen in training.

Policies in the kernel (9). FIFO, Clock, Aging, LFU; the learned linear score (`global`: one set of weights for all workloads); five heavier learned models built in as whole-number code; and ARC, LHD, S3-FIFO and LeCaR in the form a kernel can run (Section 1.1). A built-in test checks that every policy returns the program's data unchanged (checksum `AB0C8578` for all 9).

Simulator. A program that replays a recorded stream of page accesses with the same policy code (the learned score uses the kernel's own feature code). It takes seconds instead of minutes to hours, so it was used for training, for trying many variants, and for the perfect-hindsight policy (Belady) and LRU, which a kernel cannot run. On the same streams it matches the kernel's fault counts to within about 2% on average (Section 6). Appendix E lists every experiment, where it ran and where its results are.

1.1 The classical policies in the kernel

A kernel does not see every memory access. It sees faults, and whether a page was used (and written) since it last looked. ARC, LHD, S3-FIFO and LeCaR as published react to every access, so they cannot be run unchanged. Each was adapted the same way: the "used" bits are read, and cleared, at each eviction, and a page found used counts as having been used since the last look. Everything else follows the original. LeCaR's two advisers (LRU and LFU in the paper) become Clock and the kernel's "how often used" counter. The simulator versions were checked against independent reference implementations (identical faults on every trace tried). They are **not** the published algorithms; their results are results for these adapted forms.

2 Main result: every policy, three memory sizes

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.12	1.06	1.42	1.45	1.18	2.01	1.36	1.07	1.07	1.09	1.12	1.11	1.14	1.27	1.23
Aging	1.07	1.02	1.06	1.07	1.02	1.31	0.98	1.01	1.01	1.00	1.06	1.00	1.04	0.99	1.04
LFU	1.09	1.03	1.16	1.18	1.04	1.31	0.99	1.01	1.01	1.04	1.06	1.00	1.05	1.02	1.07
S3-FIFO	0.96	0.99	1.05	1.11	1.07	1.61	1.50	0.86	1.05	1.09	0.97	1.01	0.94	1.23	1.09
LeCaR	1.09	1.03	1.14	1.16	1.03	1.15	0.98	1.00	1.01	1.03	1.06	1.00	1.05	1.02	1.05
LHD	0.96	0.99	0.96	0.99	0.94	1.18	1.10	1.00	1.00	1.00	0.97	1.00	0.98	1.01	1.00
ARC	0.89	1.02	0.86	0.88	0.97	0.99	0.97	0.91	0.99	1.00	0.91	1.00	0.87	1.01	0.95
linear, global	0.84	1.01	0.82	0.83	0.95	1.72	1.03	0.86	0.98	1.10	0.88	1.00	0.87	1.04	0.98

Table 1: 5% memory. Faults relative to Clock in xv6; one column per workload (geometric mean over its streams); last column: mean over the 14 workloads.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.11	1.64	1.29	1.31	1.13	1.82	1.24	1.04	1.04	1.63	1.09	1.06	1.19	1.22	1.25
Aging	1.09	1.42	1.12	1.12	1.05	1.00	0.99	1.02	1.01	1.00	1.06	1.00	1.14	0.98	1.07
LFU	1.10	1.61	1.17	1.18	1.06	1.00	1.00	1.02	1.01	1.58	1.06	1.00	1.14	1.09	1.13
S3-FIFO	0.99	1.11	1.08	1.11	1.11	0.90	1.05	0.76	1.08	1.05	0.99	1.01	0.97	1.08	1.02
LeCaR	1.09	1.60	1.15	1.16	1.05	1.00	1.00	0.95	1.02	1.47	1.06	1.00	1.14	1.06	1.11
LHD	0.97	0.63	1.00	1.04	0.87	0.87	1.04	1.01	0.99	1.03	0.98	1.00	1.00	0.99	0.95
ARC	0.89	0.86	0.89	0.90	0.98	0.97	1.01	0.85	1.00	1.09	0.92	1.00	0.88	0.99	0.94
linear, global	0.84	0.74	0.88	0.85	0.94	1.10	0.97	0.73	0.99	1.12	0.86	1.00	0.87	1.00	0.91

Table 2: 10% memory, same layout (74 streams).

- **ARC** is the best classical policy at small memory (0.85–0.91 on btree, kv, heap and the Twitter replays at 5 and 10%).

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.11	1.74	1.23	1.25	1.60	1.58	1.14	0.90	2.38	2.11	1.08	1.03	1.12	1.31	1.35
Aging	1.10	1.56	1.14	1.15	1.45	1.11	1.00	0.88	2.29	1.03	1.06	1.00	1.10	0.98	1.17
LFU	1.10	1.71	1.17	1.18	1.55	1.11	1.00	0.93	2.35	2.09	1.07	1.00	1.10	1.19	1.27
S3-FIFO	1.05	1.38	1.14	1.15	1.61	1.11	0.99	0.68	1.08	1.04	1.02	1.01	1.00	1.19	1.09
LeCaR	1.10	1.57	1.16	1.18	1.52	0.88	1.01	0.78	2.32	1.90	1.06	1.00	1.10	1.16	1.21
LHD	0.96	0.99	1.00	1.05	0.85	0.59	1.02	1.08	0.90	1.00	0.97	1.00	1.00	0.98	0.95
ARC	0.93	1.03	0.93	0.93	0.94	1.08	1.03	1.04	1.01	1.04	0.94	1.00	0.92	0.98	0.98
linear, global	0.90	1.40	0.96	0.89	0.95	0.92	0.90	0.56	0.55	0.83	0.85	1.00	0.88	1.09	0.88

Table 3: 20% memory, same layout.

- **The linear score** does best on btree and kv (0.82–0.96 at all sizes), heap and pointer chase, and the hash join at 20% (0.55). It fails in specific places that repeat across sizes: matmul at 5% (1.72; only 2 pages of memory), SpMV at 5 and 10% (1.10, 1.12), graph at 5 and 20% (1.01 and 1.40).
- **LHD** wins clearly on graph at 10% (0.63) and on LZW and matmul at 20%, and is the most uneven.
- **LeCaR** (74 streams at each size) is **worse than Clock** at every size: 1.051, 1.113 and 1.212 of its faults at 5, 10 and 20% (intervals 1.02–1.09, 1.04–1.23, 1.05–1.43), and 1.26, 1.20 and 1.39 of its page writes; it is worst on graph and SpMV, and on the hash join at 20% (2.32).
- **No policy helps** on the Bloom filter (all 1.00) or on the application programs at 5%.
- Page writes to disk (a changed page must be written before it is thrown out) are in Appendix B.

comparison	5% memory	10% memory	20% memory
ARC against Clock	**0.947 [0.92, 0.98]**	**0.943 [0.90, 0.98]**	0.984 [0.95, 1.02]
LHD against Clock	1.005 [0.98, 1.05]	0.951 [0.88, 1.00]	0.949 [0.87, 1.01]
S3-FIFO against Clock	1.086 [1.00, 1.22]	1.016 [0.96, 1.07]	1.086 [0.98, 1.20]
linear, global against Clock	0.976 [0.90, 1.09]	**0.914 [0.85, 0.98]**	0.882 [0.77, 1.00]
linear, global against ARC	1.031 [0.97, 1.13]	0.970 [0.92, 1.02]	0.896 [0.77, 1.02]
linear, global against LHD	0.972 [0.91, 1.06]	0.961 [0.89, 1.04]	0.929 [0.80, 1.08]
LeCaR against Clock	1.051 [1.02, 1.09]	1.113 [1.04, 1.23]	1.212 [1.05, 1.43]
linear, global against LeCaR	0.929 [0.84, 1.04]	**0.822 [0.73, 0.91]**	**0.728 [0.58, 0.87]**
linear, 32 sampled against Clock	–	**0.912 [0.86, 0.97]**	–
linear, 16 sampled against Clock	–	**0.926 [0.88, 0.97]**	–
linear, 32 sampled against linear, global	–	0.998 [0.97, 1.02]	–
linear, 16 sampled against linear, global	–	1.013 [0.98, 1.04]	–
CNN against Clock	0.932 [0.86, 1.00]	**0.914 [0.84, 0.97]**	**0.829 [0.68, 0.94]**
deeper MLP against Clock	0.954 [0.87, 1.06]	**0.912 [0.84, 0.97]**	**0.838 [0.70, 0.94]**
CNN against ARC	0.999 [0.96, 1.04]	0.969 [0.90, 1.02]	0.864 [0.69, 1.02]
deeper MLP against ARC	1.024 [0.96, 1.10]	0.968 [0.90, 1.02]	0.873 [0.71, 1.01]
set-attention against ARC	**0.965 [0.94, 0.98]**	0.962 [0.89, 1.02]	0.879 [0.73, 1.03]
boosted trees against ARC	0.997 [0.97, 1.02]	1.029 [0.97, 1.09]	0.901 [0.72, 1.05]
CNN against linear, global	0.981 [0.90, 1.05]	1.000 [0.94, 1.06]	1.028 [0.94, 1.13]
deeper MLP against linear, global	1.006 [0.97, 1.04]	0.998 [0.94, 1.06]	1.039 [0.96, 1.12]
set-attention against linear, global	0.948 [0.87, 1.01]	0.992 [0.93, 1.04]	1.046 [0.95, 1.14]
boosted trees against linear, global	0.979 [0.89, 1.05]	1.061 [1.01, 1.12]	1.073 [0.99, 1.17]
ranking trees against linear, global	1.149 [0.96, 1.44]	1.476 [1.05, 2.43]	1.092 [0.99, 1.21]

Table 4: The main comparisons, with 95% intervals. Bold: the whole interval is below 1. At 10% memory every row uses all 74 streams. At 5 and 20% the heavier models (CNN, MLP, set-attention, trees) have only ten streams, and the sampled scores were run at 10% only.

Reading the intervals. They are wide because the workloads differ a lot and there are only 14. "Better" (bold) holds for: ARC against Clock at 5 and 10%; the linear score, its 32- and 16-page versions, the CNN and the MLP against Clock at 10%; the CNN and MLP against Clock at 20% and set-attention against ARC at 5% on ten streams only. "Worse" holds for LeCaR against Clock at every size, and for boosted and ranking trees against the linear score at 10%. Everything else is level or too uncertain.

3 The heavier models

Five kinds of heavier model were trained on all six training workloads and built into the kernel: a CNN over the 8-bit use history, a deeper MLP, set-attention (looks at all pages together), boosted trees and ranking trees. They ran on all 74 streams at 10% memory and on ten streams (one per variant of ten workloads) at 5 and 20%; the remaining 5 and 20% runs are prepared (Kaggle round 9) but not yet run.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.82	–	0.81	0.78	–	1.48	0.98	0.92	0.91	1.14	0.83	1.00	–	–	0.95
ARC	0.89	–	0.83	0.84	–	0.99	1.00	0.94	0.98	1.00	0.88	1.00	–	–	0.93
LHD	0.97	–	0.95	0.98	–	1.40	1.06	0.99	1.01	0.99	0.96	1.00	–	–	1.02
CNN	0.81	–	0.82	0.75	–	1.01	1.08	0.99	0.94	1.09	0.88	1.00	–	–	0.93
deeper MLP	0.81	–	0.80	0.76	–	1.29	1.01	0.99	0.95	1.19	0.88	1.00	–	–	0.95
set-attention	0.84	–	0.79	0.76	–	0.99	0.97	0.91	0.97	1.00	0.82	1.00	–	–	0.90
boosted trees	0.88	–	0.80	0.76	–	1.00	0.97	1.00	0.99	1.05	0.89	1.00	–	–	0.93
ranking trees	1.10	–	0.91	0.78	–	1.03	0.95	0.92	1.03	3.10	1.06	1.00	–	–	1.09

Table 5: 5% memory, the ten streams the heavier models have there. Some workloads are missing because they have no stream here.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.84	0.74	0.88	0.85	0.94	1.10	0.97	0.73	0.99	1.12	0.86	1.00	0.87	1.00	0.91
ARC	0.89	0.86	0.89	0.90	0.98	0.97	1.01	0.85	1.00	1.09	0.92	1.00	0.88	0.99	0.94
LHD	0.97	0.63	1.00	1.04	0.87	0.87	1.04	1.01	0.99	1.03	0.98	1.00	1.00	0.99	0.95
CNN	0.83	0.59	0.93	0.85	0.93	0.94	1.07	0.96	0.97	1.01	0.92	1.00	0.89	1.02	0.91
deeper MLP	0.82	0.59	0.93	0.86	0.91	0.94	0.98	0.96	0.99	1.05	0.92	1.00	0.87	1.06	0.91
set-attention	0.87	0.55	0.92	0.93	0.92	0.94	1.02	0.72	0.98	1.18	0.86	1.00	0.94	1.07	0.91
boosted trees	0.92	0.77	0.94	0.86	0.95	1.02	0.97	0.97	1.10	1.28	0.93	1.00	0.99	0.95	0.97
ranking trees	1.10	10.52	1.07	0.88	1.30	0.97	0.94	0.73	0.98	5.81	1.04	1.00	1.19	1.00	1.35

Table 6: 10% memory, all 74 streams.

- At 10% (74 streams) the CNN, MLP and set-attention match the linear score (1.000, 0.998, 0.992; intervals 0.93 to 1.06). At 20% (ten streams) they are 12 to 14% better than ARC, but the intervals include 1.
- **Ranking trees lose** (1.09 against Clock at 5% on ten streams; 1.35 at 10% on 74, with graph at 10.5 times Clock's faults and SpMV at 5.8 times).
- Trained on one workload instead of all, four of the five kinds are worse (simulator, faults against Clock):

model	trained on one workload	trained on all workloads
linear, global	–	0.894
CNN on aging bits	0.939	0.878
deeper MLP	0.926	0.886
boosted trees	0.981	0.963
set-attention	1.478	0.895
ranking trees	1.109	1.186

4 Looking at fewer pages (sampling)

What sampling does. Normally, at every eviction the linear score looks at every page in memory: it reads and clears the page's "used" bit, updates its counters, computes its features and its score, and throws out the highest score. With sampling it first picks k pages at random and does all of that only for those k; the victim is the best of the k. The other pages are not touched (their "used" bits keep accumulating until a later sample picks them). We measure how many more faults this causes over a whole run, and how many instructions it saves.

In the kernel ($k = 32$ and 16, random, 10% memory, 74 streams): 32 pages are level with all pages (0.998, interval 0.975 to 1.017) and beat Clock (0.912); 16 pages are 1.3% worse than all pages (1.013, 0.98 to 1.04) and still beat Clock (0.926). Other values of k, other memory sizes and other ways of choosing the pages were tried in the simulator only (below); the kernel runs for other ways of choosing are prepared (Kaggle round 8) but not run.

In the simulator, all sizes we tried (the first sweep: global linear score, 33 streams of the original six workloads at each memory size, 3 random seeds each; faults relative to looking at all pages). "Score only": every page is still scanned, only the scoring is limited to the sample. "Scan and score": only the sampled pages are scanned and scored, as in the kernel.

On these 33 streams (the original six workloads) sampling costs almost nothing at 5 and 20% memory even at $k = 8$, and 4% at $k = 16$ at 10%. **On all 74 streams the picture is less kind at 20%:** in the simulator, 16 random pages are 1.059 and 32 pages 1.031 of scoring every page at 20% memory (Table 9); the kernel runs at 5 and 20% are queued, so only the 10% claim is measured in the kernel. An earlier version of this

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.91	–	0.94	0.83	–	0.98	0.90	0.69	0.41	0.81	0.80	1.00	–	–	0.81
ARC	0.93	–	0.92	0.88	–	1.12	1.02	0.84	1.04	0.97	0.92	1.00	–	–	0.96
LHD	0.96	–	1.00	1.06	–	0.63	1.01	1.02	0.87	1.01	0.96	1.00	–	–	0.94
CNN	0.91	–	1.05	0.83	–	0.72	0.89	0.96	0.39	0.89	0.90	1.00	–	–	0.83
deeper MLP	0.90	–	1.02	0.83	–	0.80	0.89	0.94	0.40	0.93	0.90	1.00	–	–	0.84
set-attention	0.97	–	1.03	1.01	–	0.72	0.88	0.68	0.46	1.10	0.84	1.00	–	–	0.84
boosted trees	1.00	–	1.03	0.84	–	0.82	0.91	0.97	0.39	1.04	0.91	1.00	–	–	0.86
ranking trees	1.26	–	1.13	0.88	–	0.86	0.85	0.69	0.38	1.06	1.09	1.00	–	–	0.88

Table 7: 20% memory, ten streams.

pages looked at	5%	10%	20%	5%	10%	20%
4	1.028	1.124	1.045	1.035	1.136	1.063
8	1.009	1.071	1.000	1.011	1.080	1.008
16	1.000	1.036	1.001	1.001	1.040	1.001
32	0.997	1.016	0.999	0.998	1.018	0.999
64	0.999	1.006	0.999	0.999	1.007	0.999

Table 8: Faults with k sampled pages divided by faults with all pages (simulator). First three columns: score only; last three: scan and score (the kernel's way). The three seeds differ by under 0.002.

report quoted 1.017 and 1.04 for $k = 32$ and 16 from this sweep; on the 74 kernel streams the simulator gives 1.002 and 1.019 and the kernel 0.998 and 1.013. The difference is the choice of streams (those 33 streams alone give 1.016 and 1.039), not the simulator.

Which pages to sample (simulator, the kernel's 74 streams and memory sizes at 10%, the kernel's linear score, scan and score):

which pages	32: vs all pages	16: vs all pages	32: vs Clock	16: vs Clock
all pages (no sampling)	1.000	1.000	0.912	
random (as in the kernel runs)	1.002	1.019	0.914	0.929
the quarter newest loaded, rest random	1.002	1.021	0.913	0.931
evenly spread over the load order	1.015	1.019	0.925	0.929
a window in load order that moves on	1.056	1.086	0.963	0.990
pages not seen at their latest scan first	1.058	1.089	0.964	0.993
the oldest loaded pages	1.149	1.198	1.048	1.092

Random is as good as anything tried. Taking the newest pages plus random ones ties it; an even spread ties it at 16 but at 32 it becomes "the 32 oldest pages" whenever fewer than 64 pages are in memory, which is worse; the others are clearly worse. Measured exactly in the kernel with 68 pages, random sampling costs 11 305 (32) and 5 668 (16) instructions, the even spread 11 275 and 5 681 (the random draws cost almost nothing, so it saves nothing), and newest-plus-random 10 759 and 5 002. On one stream (kv-T17) the kernel and simulator agree to within 1–3% for every scheme. Not done: kernel runs with several random seeds (the seed check is from the simulator only).

Looking for a cheaper rule (simulator, all 74 streams at 5, 10 and 20% memory, scan and score). Two ideas beyond picking pages at random, aimed at fewer pages per eviction: *pool* keeps the best few candidates of the last eviction and adds k fresh random pages (what Redis does, so a good candidate is not forgotten), and *clock-filter* walks the pages in load order like Clock, clears their used bits and scores only the first k found unused. Each run records the pages scanned and scored per eviction; the instructions are estimated from the kernel's exact counts (264 per sampled page: 118 to scan, 146 to score; a fit to the counts for 4 to 64 random pages, which it reproduces within 2%; bookkeeping of the pool is not counted).

At equal estimated cost, pool is level with random or at most about 10% cheaper (pool 16+16 is 1.004 at about 6 600 instructions against random 32 at 1.002 and 7 400); clock-filter is clearly worse than random at every size (1.062 at about 4 300 against random 16 at 1.019 and 4 000). So the quality-against-cost curve of random sampling is close to the best we found: about 16 pages is the knee at 5 and 10% (1.001 and 1.019 of all pages at about a quarter of the cost), and at 20% memory more pages are needed.

scheme	pages scored per eviction (10%)	estimated instructions (10%)	5%	10%	20%
random 4	4	1 082	1.053	1.116	1.213
random 8	8	2 092	1.018	1.054	1.117
random 16	15	3 960	1.001	1.019	1.059
random 32	28	7 367	0.996	1.002	1.031
pool 4 new + 4 kept	8	2 015	1.020	1.063	1.127
pool 4 new + 8 kept	11	2 913	1.016	1.054	1.111
pool 8 new + 8 kept	14	3 743	1.002	1.017	1.056
pool 8 new + 16 kept	20	5 308	1.003	1.015	1.051
pool 16 new + 16 kept	25	6 640	1.000	1.004	1.028
clock-filter 4	4	1 331	1.036	1.061	1.134
clock-filter 8	8	2 378	1.026	1.074	1.121
clock-filter 16	15	4 266	1.010	1.062	1.109
clock-filter 32	28	7 697	1.000	1.040	1.096

Table 9: Faults with sampling divided by faults with all pages (simulator), and the estimated instructions per eviction at 10% memory.

5 What choosing a victim costs

5.1 Instructions, on one synthetic benchmark

The exact number of instructions the kernel spends choosing one victim was counted with QEMU's instruction-counting mode, in which the kernel's timer advances exactly once per 100 instructions (repeatable to under 0.3%). These counts come from **one benchmark**: `patbench` reading pages uniformly at random, chosen so that every eviction has exactly the same number of pages to choose from and every policy sees the same accesses. We measured four memory limits (16, 64, 256, 512 pages, so 20, 68, 260 and 516 pages to choose from):

policy	20 pages	68 pages	260 pages	516 pages	per extra page
FIFO	144	384	1 345	2 624	5
Clock	403	410	645	1 068	1
Aging	2 460	8 124	30 776	60 978	118
LFU	2 435	8 004	30 279	59 980	116
S3-FIFO	759	1 395	3 726	6 452	11
ARC	4 499	11 934	41 451	81 070	154
LHD	5 086	14 397	54 630	111 705	217
LeCaR	2 538	8 156	30 658	60 678	117
linear score, all pages	4 568	16 818	67 163	134 392	262
linear score, 32 sampled	4 570	8 465	8 788	8 880	1
linear score, 16 sampled	3 960	4 237	4 400	4 457	0
boosted trees	135 762	455 229	1 750 016	3 516 040	6 832
ranking trees	127 830	440 490	1 681 339	3 321 130	6 430
set-attention	534 288	1 801 153	6 798 764	13 460 877	26 026
CNN	569 615	1 936 230	7 405 413	14 697 650	28 485
deeper MLP	1 004 031	3 413 317	13 052 592	25 905 505	50 206

Table 10: Instructions to choose one victim, by number of pages to choose from (patbench uniform). Last column: extra instructions per extra page, from 68 to 516 pages.

The cost of the linear score, Aging, LFU, ARC, LHD, LeCaR and the heavier models grows with every page in memory (the linear score about 260 instructions per page in the current kernel, 340–360 before the code change: it walks the page table to read the "used" bit, updates the counters, computes five features, four of them logarithms done in whole numbers, and adds up the weighted features). Clock and S3-FIFO usually stop after a few pages, so they barely grow. The sampled scores stop growing once memory holds more pages than the sample. A single number such as "16 800 instructions" is only the 68-page value. The table is from the kernel with the cheaper linear-score code; before it, the linear score cost 22 803, 91 720 and 183 311 instructions at 68, 260 and 516 pages (`cost_grid_before_A.csv`). Other policies moved by at most a few percent from the change of build (the networks 2–3% lower, LHD 1.4% higher); every fault count in this report is from before the change, and was checked to be identical after it (Section 5.4).

5.2 A rough check on the real programs

Every kernel run also recorded the time spent choosing victims and how many pages there were to choose from. These runs were not in instruction-counting mode, so the time is emulated time and depends on the PC's load; to cancel the machine's speed, each policy is divided by Aging on the same streams (Aging scans every page and does little else). Runs on the thesis PC only, 10% memory:

workload	pages to choose from	Clock	LeCaR	ARC	LHD	linear
patbench uniform (exact count)	68	0.05	1.00	1.47	1.77	2.07
app	43	0.31	1.44	2.09	2.17	2.56
bloom	55	0.20	1.30	2.75	2.97	2.74
btree	221	0.07	1.11	1.66	2.07	2.85
chase	106	0.14	1.35	3.07	3.56	2.92
graph	170	0.16	1.37	1.79	2.26	3.05
heap	48	0.22	1.28	1.85	2.09	2.75
join	132	0.14	1.42	3.21	3.97	2.93
kv	57	0.25	1.35	2.38	2.59	2.66
kv-T	56	0.27	1.31	2.14	2.35	2.64
lzw	49	0.29	1.63	2.51	2.77	2.68
matmul	9	0.74	1.58	3.37	3.08	1.88
pat	106	0.14	1.28	1.71	1.98	2.85
sort	16	0.46	1.49	3.10	2.91	2.03
spmv	171	0.11	1.05	1.61	1.71	2.64

Table 11: Time spent choosing victims relative to Aging, median over each workload's streams (rough; `tools/ml2/cost_from_runs.py`). First row: the exact patbench ratio.

For the linear score patbench is fair: 2.6 to 3.1 times Aging on every workload with more than 40 pages, against 2.81 on patbench (lower on matmul and sort, which have only 9 and 16 pages, so the fixed part of each eviction weighs more). For ARC and LHD it is not: on the real programs they cost 1.6 to 4.0 times Aging, against 1.47 and 1.75 on patbench, so the patbench counts underestimate them by up to about 2 times, and Clock and LeCaR are also somewhat dearer. Exact counts on one stream of each workload are planned; until then the costs of ARC and LHD in this report are too low.

5.3 From instructions to time

Disk reads, measured on the thesis PC (4 KB random reads, one at a time, file cache off): a Samsung 980 NVMe SSD, median 63 µs (99th percentile 134 µs); a WD Blue 1 TB hard disk, median 8.9 ms (99th percentile 24.5 ms; reads spread over a 1 GB file, a pessimistic figure for a small swap area). These are bare disk reads, not whole xv6 page faults. **The scoring code's speed**: the kernel's own scoring code compiled for this PC's processor (one core of a Ryzen 5 5600), 68 pages, microseconds per decision:

policy	scoring only	+ page-table walk	+ walk, cold caches
linear score (global)	1.35	1.52	2.11
linear score (global) 32 sampled	0.86	0.89	1.45
linear score (global) 16 sampled	0.43	0.45	0.91
boosted trees	32.52	31.78	33.85
ranking trees	30.11	27.93	29.80
set-attention	105.77	107.94	108.09
CNN	106.94	106.47	109.97
deeper MLP	206.19	200.91	205.03

That is about 0.05 to 0.08 ns per kernel instruction (provisional for the new code: this timing was taken while other jobs were running; the table above is the older, quiet measurement of the code before the change; the new code ran at 0.63 of the old code's time, 947 against 1 495 ns under the same load, `native_timing_A_under_load.csv`). It leaves out the locks and the flush of the address-translation cache after the scan, so it is a lower bound, and it runs on an x86 processor, not RISC-V.

Does the extra work pay? The linear score spends about 16 400 more instructions per eviction than Clock (about 1 µs here, provisional) and saves about 9% of the faults. On the SSD each eviction then saves about $0.09 \times 63 \approx 5.7$ µs of disk time, more than it spends; on the hard disk about 800 µs. On a slow processor (1 ns per instruction) it spends about 16 µs, which still does not pay on the SSD for the full score (1.23 of Clock's time; the 16-page sample, 0.98, does). Time on faults plus choosing, relative to Clock (geometric mean over workloads; 10% memory; using the patbench instruction counts):

policy	faults / Clock	SSD, 1 ns	disk, 1 ns	SSD, 0.059 ns	disk, 0.059 ns
linear score, all candidates	0.914	1.23	**0.92**	**0.93**	**0.91**
linear score, 32 sampled	0.912	1.02	**0.91**	**0.92**	**0.91**
linear score, 16 sampled	0.926	**0.98**	**0.93**	**0.93**	**0.93**
ARC	0.943	1.17	**0.94**	**0.95**	**0.94**
LHD	0.951	1.23	**0.95**	**0.97**	**0.95**
LeCaR	1.113	1.30	1.11	1.12	1.11
S3-FIFO	1.016	1.04	1.02	1.02	1.02
CNN	0.914	29.27	1.18	2.58	**0.93**
deeper MLP	0.912	50.42	1.36	3.73	**0.94**
set-attention	0.907	27.17	1.15	2.45	**0.92**
boosted trees	0.970	8.33	1.04	1.43	**0.97**
ranking trees	1.350	11.33	1.44	1.97	1.35

SSD 63 µs per read, hard disk 8.9 ms (measured). "1 ns": a slow processor; "0.051 ns": this PC's processor with the cheaper code (provisional, see above). Bold: faster than Clock. The CNN, MLP and set-attention do not pay on an SSD, and win only a few percent on a hard disk with a fast processor. The per-workload break-even is in Appendix A. This counts only disk time per fault and choosing a victim; the rest of a fault and the program's own work are left out.

5.4 Reducing the instruction count

Compiler settings. Every run used the kernel's normal setting (`-O`). Exact counts with other settings (patbench, 68 pages):

kernel built with	Clock	Aging	linear, all pages	linear, 32 sampled
-O (what every run used)	410	8 123	22 803	11 292
-O2	427	8 400	22 655	11 228
-O2 without frame pointer	411	7 992	20 379	10 150
-O3	289	6 487	20 224	10 078
-Os (smallest code)	504	8 881	26 766	13 171

`-O3` cuts the linear score by 11% and Aging by 20%; `-Os` makes everything dearer. These are not applied: a different build changes every result slightly, so all runs would have to be repeated.

Code changes that pick exactly the same victims (asked for by the paper side). We changed only the linear score's code and counted again; in every row the fault count is identical to the original, so the victims are unchanged, and the other policies cost the same:

change	68 pages	260 pages	68 pages, 32 sampled
none (what every run used)	22 803	91 720	11 292
A1: store the two logs that rarely change	21 608 (-5%)	87 506 (-5%)	10 691 (-5%)
A2: one pass instead of two	22 986 (+1%)	90 169 (-2%)	11 374 (+1%)
A3: feature code inlined	18 193 (-20%)	72 862 (-21%)	9 132 (-19%)
A1 + A3	17 174 (-25%)	69 260 (-24%)	8 620 (-24%)
A1 + A3 + A4: faster highest-bit search in the log	16 817 (-26%)	67 163 (-27%)	8 466 (-25%)
A1 + A3 + A5: features written out one by one	18 595 (-18%)	75 277 (-18%)	9 307 (-18%)
A1 + A2 + A3	18 027 (-21%)	71 531 (-22%)	9 016 (-20%)
A1 + A2, page code at -O3 (also speeds up other policies)	19 088 (-16%)	76 034 (-17%)	9 501 (-16%)

The biggest saving is A3: with the kernel's `-O` setting the feature function is called as a separate function five times per page; inlining it removes about 70 instructions per page. A1 (keep the logarithms of the two counters that change only when a page comes back from disk) and A4 (find the highest bit of a number with a byte table instead of six tests; checked to give the same logarithm for 67 million inputs) add about 6%. A2 and A5 make it slightly worse and are dropped. The best version (A1 + A3 + A4) costs 16 800 instead of 22 800 instructions with 68 pages, and 8 500 instead of 11 300 with 32 sampled. **It is now in the kernel** (2026-10-15). Checks: the data test passes for all 9 policies (checksum `AB0C8578`, no fallbacks); 28 runs (the linear score and its 32-page version on one stream of each workload, 10% memory) give exactly the same faults, zero-fill faults, evictions and page writes as before; the same code natively picks the same victim on 40 000 random decisions. The instruction table above is measured again in the new build.

6 Does the simulator predict the kernel?

Kernel faults divided by simulator faults on the same streams and memory sizes (10% memory):

policy	streams	geometric mean	median stream
ARC	74	0.995	1.005
S3-FIFO	74	1.014	1.015
LHD	74	0.984	1.000
LeCaR	59	1.000	1.004
linear score, all pages	74	1.000	0.999
linear score, 32 sampled	74	0.996	0.998
linear score, 16 sampled	74	0.993	0.999

The simulator starts from filled memory, as the kernel's benchmarks do (started from empty memory, as the first versions did, some models were misjudged, set-attention by up to 14 times). The kernel's own fixed pages (code, stack) and sizes near a sudden change in faults explain most of the remaining differences. Single streams can differ more (0.71 to 1.72 for the linear score).

7 Limits and open problems

- **One operating system:** xv6 on QEMU, one benchmark process at a time. Nothing here says what happens in Linux.
- **Instruction counts come from one synthetic benchmark;** the rough check shows ARC and LHD are dearer on real programs. Exact counts on the real programs: planned.
- **Time is measured only in parts:** exact instruction counts, a bare disk read, and the scoring code's speed on an x86 processor. A whole xv6 page fault on real hardware was never timed.
- **Fourteen workloads,** some close variants of each other; the heavier models have only ten streams at 5 and 20% memory (round 9 prepared).
- **Adapted forms:** ARC, LHD, S3-FIFO and LeCaR ran in the form a kernel can run, not as published.
- **"All workloads" means the six training workloads;** the other eight were never seen in training.
- **Sampling:** only random sampling at $k = 32$ and 16, at 10% memory, ran in the kernel, with one random seed.
- **The cheaper linear-score code** (Section 5.4) is in the kernel only since 2026-10-15; every fault count here comes from builds before it (checked identical on 28 runs, one stream per workload for the linear score and its 32-page version; not on every stream). Its native timing is provisional.
- **The learned policy can be worse than Clock** (matmul at 5%, SpMV, graph at 20%); the tables show where.

A Break-even by workload

The fault cost above which a policy beats Clock on one workload, at 10% memory, on this PC's processor (about 0.051 ns per instruction, provisional): the extra instructions spent choosing a victim divided by the

faults saved per eviction. Compare with 63 µs (SSD) and 8.9 ms (hard disk). "never": the policy does not save faults there. Uses the patbench instruction counts, so ARC is probably too favourable.

workload	linear score faults	linear score break-even μs	ARC faults	ARC break-even μs	CNN faults	CNN break-even μs
app	1.004	37.8	0.991	59.6	1.016	6 044
bloom	0.999	790	1.0	never	1.0	465 577
btree	0.841	21.2	0.894	22.4	0.831	2 172
chase	0.864	7.2	0.922	8.2	0.921	1 326
graph	0.736	5.1	0.858	2.9	0.59	493
heap	0.735	3.2	0.853	3.6	0.955	2 807
join	0.987	20.2	0.996	47.8	0.972	12 332
kv	0.879	6.0	0.892	4.3	0.932	1 535
kv-T	0.846	4.1	0.9	4.0	0.851	513
lzw	0.941	10.0	0.982	24.7	0.93	984
matmul	1.104	never	0.968	2.0	0.936	240
pat	0.873	9.3	0.877	7.0	0.895	973
sort	0.967	3.9	1.014	never	1.072	733
spmv	1.124	never	1.086	never	1.007	never

Table 12: Faults against Clock and break-even fault cost, per workload.

B Page writes

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.10	1.10	1.56	2.85	2.06	2.64	1.15	1.77	1.06	11.11	1.33	1.12	1.24	3.04	1.81
Aging	1.01	1.04	1.12	1.22	1.05	1.02	0.97	1.03	1.00	0.99	1.14	1.00	1.07	1.00	1.05
LFU	1.01	1.04	1.26	1.67	1.08	1.02	0.98	1.04	1.00	10.72	1.15	1.00	1.08	1.20	1.30
S3-FIFO	1.04	1.04	1.07	1.75	1.57	1.25	1.27	0.58	1.05	10.71	1.02	1.02	1.00	2.35	1.37
LeCaR	1.01	1.04	1.22	1.56	1.08	0.88	0.98	1.03	1.00	9.17	1.14	1.00	1.07	1.12	1.26
LHD	0.99	1.01	0.94	0.89	0.84	0.99	1.11	0.92	1.00	1.00	0.93	1.00	0.98	1.03	0.97
ARC	0.99	1.00	0.69	0.71	0.93	0.86	0.97	0.63	1.00	1.11	0.87	1.00	0.87	1.16	0.90
linear, global	0.97	1.02	0.60	0.72	1.15	2.64	0.99	0.73	1.04	20.11	0.77	1.00	0.80	1.28	1.22

Table 13: 5% memory: page writes to disk relative to Clock, same layout as the fault tables.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.06	1.87	1.60	2.21	1.74	13.00	1.12	1.51	1.02	2.25	1.28	1.07	1.32	1.40	1.69
Aging	1.02	1.52	1.31	1.33	1.14	0.99	0.99	1.06	1.00	0.99	1.19	1.00	1.22	0.97	1.11
LFU	1.02	1.75	1.42	1.57	1.17	0.99	1.00	1.07	1.00	2.20	1.20	1.00	1.23	1.02	1.22
S3-FIFO	1.03	1.13	1.26	1.67	1.63	1.15	1.01	0.60	1.05	2.22	1.06	1.02	0.97	1.18	1.16
LeCaR	1.02	1.69	1.37	1.48	1.16	0.99	1.01	1.04	1.01	2.02	1.18	1.00	1.21	1.02	1.20
LHD	0.99	0.29	1.03	1.01	0.79	0.84	1.04	0.99	0.99	0.99	0.96	1.00	0.98	0.98	0.89
ARC	0.98	0.47	0.81	0.95	0.96	0.97	1.00	0.68	1.00	1.07	0.90	1.00	0.81	1.00	0.88
linear, global	0.95	0.73	0.78	0.96	1.30	4.75	0.96	0.72	1.05	4.22	0.73	1.00	0.76	0.90	1.12

Table 14: 10% memory: page writes relative to Clock.

C The first version of the training data, and DAgger

How a model is trained. In the simulator we replay a training program while a simple rule (LRU) makes the decisions. At some evictions we take a snapshot: a set of candidate pages, each with its features and the right answer (how soon it will be needed again; the page needed last is the one the perfect policy would throw out). The model learns from many such snapshots.

What went wrong. In the first version each snapshot held 32 candidates drawn at random from the pages in memory (on PageRank at 10%: 32 of 167). The page the program is using right now is one of 167, so it was rarely in a snapshot, and the model hardly ever saw the example "just loaded, needed again very soon: keep it". When it ran, it kept throwing out the page it had just loaded. Rerun with the current seeds (`tools/ml2/r3_first_version.py`): 203 of the 300 linear models make at least 3 times Clock's faults on PageRank (runs stop there); for a failing model 86–87% of the pages it threw out had been loaded at the previous fault and were needed again a median 15–16 accesses later, while the perfect policy's choice had last been used about 230–270 accesses earlier. The best of the 300 does not fail (0.76 of Clock).

The fix. Every snapshot now also holds the 4 most recently used pages, and a page loaded in the last few checks cannot be thrown out while an older one is available ("probation", like Clock's second chance).

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.07	20.08	1.46	1.41	1.67	7.53	1.07	1.33	2.09	1.10	1.24	1.03	1.27	1.41	1.81
Aging	1.04	13.45	1.31	1.13	1.37	1.05	1.00	1.07	1.94	1.10	1.19	1.00	1.22	1.00	1.39
LFU	1.04	19.23	1.37	1.21	1.48	1.05	1.01	1.10	2.05	1.08	1.20	1.00	1.23	1.16	1.47
S3-FIFO	1.04	9.48	1.31	1.29	1.50	5.25	0.99	0.84	1.26	0.96	1.11	1.02	1.02	1.33	1.46
LeCaR	1.04	13.58	1.33	1.17	1.38	0.91	1.01	1.04	2.00	1.10	1.18	1.00	1.21	1.14	1.39
LHD	0.97	0.66	1.04	1.04	0.82	0.73	1.02	1.02	0.96	0.98	0.95	1.00	1.00	1.00	0.94
ARC	0.98	2.57	0.93	0.98	0.98	1.04	1.02	0.87	0.97	1.07	0.91	1.00	0.86	0.98	1.04
linear, global	0.95	8.67	0.93	0.94	0.99	1.89	0.90	0.63	0.59	0.51	0.65	1.00	0.75	0.72	0.98

Table 15: 20% memory: page writes relative to Clock.

DAgger. The snapshots are taken while LRU decides, but the model then meets the situations its own decisions create. DAgger runs the model, takes snapshots of those situations (still labelled by the perfect policy), adds them to the training data and retrains; one round per repeat. Three rounds on the first-version data, PageRank, faults relative to Clock:

model	round 0	round 1	round 2	round 3
best of 300	0.76	0.77	0.76	0.76
median of 300	3.00 (stopped)	2.00	2.49	3.00 (stopped)
all features	3.00 (stopped)	0.91	0.88	0.87

DAgger rescued the model that uses all features, changed nothing for the good model, and did not settle the median one. The earlier statement "three DAgger rounds did not help" is therefore not correct as a general statement.

D Simulator results

scope	split	LRU	ARC	S3F	LHD	linear K	tree K	NN K	NN K†	tree FK	NN FK	Belady
btree	test	0.994	0.950	0.862	0.948	0.835	0.834	0.822	0.817	0.829	0.811	0.550
btree	held-out	0.997	0.975	0.908	0.974	0.875	0.867	–	0.851	0.853	0.847	0.615
graph	test	0.990	0.991	1.161	0.865	0.847	0.847	–	0.844	0.806	0.844	0.456
graph	held-out	0.992	0.993	1.256	0.816	0.798	0.765	–	0.760	0.742	0.760	0.482
kv	test	0.984	0.986	1.047	0.906	0.897	0.902	0.850	0.849	0.875	0.843	0.559
kv	held-out	0.982	0.982	1.013	0.901	0.902	0.903	0.845	0.858	0.900	0.854	0.577
lzw	test	0.997	0.968	0.889	0.892	0.821	0.933	–	0.809	0.933	0.803	0.466
lzw	held-out	0.981	0.957	0.866	0.887	0.813	1.068	–	0.841	1.068	0.812	0.409
matmul	held-out	0.879	0.875	1.508	1.036	0.671	0.803	0.868	0.868	0.899	0.868	0.617
sort	test	1.012	1.013	1.321	1.008	0.983	0.879	–	0.891	0.854	0.901	0.830
sort	held-out	0.999	0.999	1.084	1.019	0.933	0.928	–	0.961	0.938	0.924	0.896
global	test	0.992	0.948	0.978	0.926	0.877	0.897	–	–	0.856	0.856	0.591
global	held-out	0.979	0.968	1.020	0.926	0.896	0.894	–	–	0.854	0.847	0.558

Table 16: Faults relative to Clock in the simulator on the training workloads' test and held-out streams: classical policies, the linear model, trees and networks over kernel-visible features (K) and with features only a simulator can see (FK), and Belady (the best possible).

stream	LRU	ARC	S3-FIFO	LHD	Belady	linear K+	linear F+K	linear kv K+
app-bloom	1.000	0.989	0.962	0.998	0.635	0.930	0.926	–
app-chase	0.991	1.002	1.021	0.984	0.683	1.021	1.020	–
app-hashjoin	0.998	1.000	1.012	0.994	0.544	1.138	1.017	–
app-spmv	0.992	1.000	1.026	1.046	0.727	1.239	0.995	–
kv-T	0.983	0.983	1.028	0.916	0.611	–	–	0.980
pat-loop	1.000	1.000	1.000	1.000	0.882	0.894	0.887	–
pat-phase	0.968	0.887	0.899	0.876	0.552	0.899	2.644	–
pat-phaseshort	0.967	0.888	0.909	0.879	0.552	0.918	2.686	–
pat-scanhot	0.992	0.786	0.795	0.866	0.700	0.748	0.744	–
pat-scanhotlo	1.001	0.870	0.874	0.982	0.803	0.829	0.903	–
pat-switch	1.005	0.882	0.877	1.007	0.704	0.817	0.805	–
pat-uniform	1.000	1.000	1.000	1.000	0.658	1.000	1.000	–
pat-zipf060	0.998	0.915	0.881	0.975	0.612	0.845	0.824	–
pat-zipf080	0.994	0.874	0.837	0.932	0.587	0.796	0.773	–
pat-zipf099	0.990	0.846	0.806	0.897	0.565	0.764	0.737	–
pat-zipf120	0.984	0.825	0.783	0.863	0.544	0.737	0.710	–
redis-real	0.929	0.967	1.104	0.973	0.667	1.106	3.000	–
sqlite-real	0.908	0.947	1.082	0.927	0.567	0.934	1.318	–

Table 17: Faults relative to Clock per stream group on workloads no model was trained on (simulator, geometric mean over streams and the three memory sizes).

scope	split	LRU	ARC	LHD	LeCaR, as published	LeCaR, kernel form	linear K
btree	test	0.994	0.950	0.948	0.906	1.122	0.835
btree	held-out	0.997	0.975	0.974	0.910	1.065	0.875
graph	test	0.990	0.991	0.865	0.981	1.327	0.847
graph	held-out	0.992	0.993	0.816	0.968	1.321	0.798
kv	test	0.984	0.986	0.906	0.904	1.127	0.897
kv	held-out	0.982	0.982	0.901	0.918	1.149	0.902
lzw	test	0.997	0.968	0.892	1.043	1.056	0.821
lzw	held-out	0.981	0.957	0.887	1.042	1.190	0.813
matmul	held-out	0.879	0.875	1.036	0.851	0.882	0.671
sort	test	1.012	1.013	1.008	0.980	1.016	0.983
sort	held-out	0.999	0.999	1.019	1.012	1.000	0.933
global	held-out	0.979	0.968	0.926	0.958	1.116	0.896

Table 18: LeCaR in the simulator. "As published": advisers exact LRU and exact LFU (needs every access, so a kernel cannot run it). "Kernel form": Clock and the kernel's "how often used" count, read at evictions. Original parameters (learning rate 0.45, history of half the memory per adviser).

LeCaR picks one of two advisers at random at each eviction, in proportion to how much it trusts each, and remembers which adviser threw out each page; when a thrown-out page is needed again soon, that adviser loses trust. As published it does well on btree and kv (0.90 to 0.92) and ties ARC and LRU on graph; in the kernel form it is worse than Clock in the simulator (1.06 to 1.33) and in the kernel (1.05, 1.11 and 1.21 at 5, 10 and 20%, 74 streams).

E Every experiment

experiment	where	size	results
recording and checking the program streams	xv6 + checks	199 streams	`traces2/` (not in git)
classical policies incl. LRU and Belady	simulator	every stream, 6 sizes	`classical.csv`, `baselines_ext.csv`
hand-written rules	simulator		`summary_handwritten.csv`
linear models, every feature set, first version	simulator	28 800 replays	`linear_sweep_v1.csv`; App. C
linear models, every feature set, current	simulator	28 800 replays	`linear_sweep.csv`
neural networks and trees	simulator (GPU training)	84 networks + trees	`extra_summary.csv`
trained on one vs all workloads; unseen workloads	simulator		`trainedall`, `transfer.csv`
whole-number scoring, probation, DAgger (1 round)	simulator		`extras.csv`
training-sample noise	simulator		`noise_floor.csv`
first version and DAgger (3 rounds), rerun	simulator	300 models, 1 stream	`r3_first_version.*`, `r3_dagger_v1.csv`
sampling, $k = 4$ to 64	simulator	33 streams × 3 sizes × 3 seeds	`sampling_sweep.csv`
ways of choosing the sample	simulator	74 streams, 10%	`sampling_heuristics.csv`
LeCaR	simulator		`baselines_ext.csv`
every policy and model in the kernel	xv6 (PC + Kaggle rounds 1–6)	3 405 runs	`kernel_eval_all.csv`
LeCaR in the kernel	xv6 (PC)	74 streams × 3 sizes	`kernel_eval_lecar.csv`
kernel against simulator	both	74 streams	`kernel_vs_sim*.csv`
instruction counts (cost grid)	xv6, counting mode	28 policies × 4 sizes	`cost_grid.csv`
time spent choosing, real programs (rough)	from the xv6 runs	2 764 PC runs	`cost_from_runs.csv`
scoring code speed	this PC's processor		`native_timing*.csv`
disk read time	this PC (Windows)	SSD, hard disk	`device_latency.csv`
time and break-even	computed		`cost_benefit_*.csv`
compiler settings	xv6, counting mode	5 builds	`compiler_flags.csv`
sampling schemes in the kernel (cost; 1 stream)	xv6		Section 4
prepared, not run: sampling schemes on 74 streams	Kaggle round 8 + PC	444 runs	
prepared, not run: heavier models, 5 and 20%, 74 streams	Kaggle round 9 + PC	540 runs	
planned: exact instruction counts on the real programs	xv6, counting mode	14 workloads	

F How to reproduce

- Kernel runs: `python3 tools/ml2/kernel_eval.py -fracs 0.1 -nonml clock,arck -models global,global@32`; results go to `report/results2/`; the Kaggle packages are in `kaggle_package/round1..round9` (see `INDEX.txt`).

- Merge: `merge_kernel_results.py`; intervals: `kernel_ci_classic.py` (and `writes`); tables: `report_w15.py`, then `report_w16.py`.
- Costs: `cost_grid.py` (counting mode), `cost_from_runs.py`, `native_timing.c`, `tools/io_latency.ps1`, `cost_benefit.py`, `flags_test.py`.
- Simulator studies: `exp_sampling.py`, `exp_sampling_heur.py`, `r3_first_version.py`, `r3_dagger_v1.py`.